CS253

Assignment 1

Memory-Efficient Versioned File Indexer

Roll Number: 240234 | Name: Avtansh Gargya

1. Overview

The goal of this assignment was to build a word-frequency indexer that can process large log files without ever loading them fully into memory. The approach is to read the file in fixed-size chunks using a heap-allocated buffer and accumulate word counts incrementally. The result is a tool that can handle files of any size and supports three types of queries: looking up the frequency of a specific word, listing the most frequent words, and comparing frequencies across two different file versions.

2. Datasets

Two log files were provided for this assignment, each about 109 MB in size.

File	Log Format	Unique Words
verbose_logs.txt	[LEVEL] At timestamp N, module <Service> reported: ...	439,301
test_logs.txt	LEVEL <Service>: <short message>	33

The verbose log file uses full sentence descriptions with service names and timestamps, which explains its large vocabulary. The test log file is much more compact and repetitive — the same handful of words appear over and over, which is why it has only 33 unique words despite being the same file size. Both files were opened read-only and not modified in any way.

3. Design

The code is split into nine classes. Each one has a specific job and they're composed together in main() to handle whichever query the user runs.

Class	Job
FrequencyMap<K,V>	Generic word-to-count map, templated on key and value type
BufferedFileReader	Opens a file and fills a fixed buffer one chunk at a time
Tokenizer	Pulls lowercase alphanumeric words out of a buffer; handles split words
VersionedIndex	One FrequencyMap per named version; runs the read/tokenise loop
Query	Abstract base — WordQuery, TopKQuery, DiffQuery all inherit from this
WordQuery	Looks up a single word's count in a version
TopKQuery	Returns the K most frequent words in a version
DiffQuery	Compares a word's count between two versions
QueryProcessor	Calls execute() on whatever Query is passed to it

4. Class Details

4.1 FrequencyMap

A simple template wrapper around unordered_map. Templating it makes it reusable for any key/value combination, though in practice it's always instantiated as FrequencyMap. It exposes increment(), get(), getAll(), and size().

4.2 BufferedFileReader

Allocates a single char array of bufferSizeKB * 1024 bytes on the heap and fills it by calling file_.read() in a loop. Once the buffer is exhausted it fills again, overwriting the previous chunk. The constructor validates that the requested buffer size is in the 256-1024 KB range and throws invalid_argument if not. Copy construction is deleted to avoid double-freeing the buffer.

4.3 Tokenizer

The tricky part of chunked file reading is that a word can be cut in half at a chunk boundary — for example 'OrderService' might appear as 'OrderSer' at the end of one chunk and 'vice' at the start of the next. The Tokenizer keeps a leftover_ string for exactly this case: any incomplete word at the end of a chunk is saved and prepended to the next one. There are two overloads of tokenize() — one that works on a raw buffer with a callback, and one that works on a plain string and returns a vector. The second is used to normalise the query word from the command line.

4.4 VersionedIndex

Owns an unordered_map from version names to FrequencyMap instances. addVersion() creates a BufferedFileReader and Tokenizer locally, loops until the file is fully read, and feeds every token into the corresponding frequency map. Since nothing accumulates beyond the word counts themselves, memory usage scales with the number of unique words, not with file size.

4.5 Query, WordQuery, TopKQuery, DiffQuery

Query defines a pure virtual execute() method. The three derived classes each implement it differently — WordQuery does a single map lookup, TopKQuery sorts the full map and takes the top K entries, and DiffQuery does two lookups and subtracts. QueryProcessor calls execute() on whatever Query object it receives, so the right implementation is selected at runtime via the vtable. It's also overloaded to accept either a reference or a pointer to a Query.

5. C++ Requirements

Requirement	How it's done
4+ user-defined classes	9 classes total
Abstract base + 2 derived	Query -> WordQuery, TopKQuery, DiffQuery
Runtime polymorphism	processor.process() calls execute() through the vtable
Function overloading	Tokenizer::tokenize() has two signatures; QueryProcessor::process() accepts ref or pointer
Exception handling	try/catch in main; throw in constructor and parseArgs
User-defined template	FrequencyMap<K, V>

6. Memory

Three things keep memory bounded regardless of file size:

The buffer is a single fixed allocation that gets reused every chunk.

The Tokenizer's leftover_ holds at most one partial word at a time.

VersionedIndex only stores one entry per unique word — not one per occurrence.

On verbose_logs.txt (109 MB, 439k unique words) the index takes roughly 30-40 MB of heap. On test_logs.txt (same size, 33 unique words) it's negligible. Neither is anywhere close to the file size.

7. Results

7.1 Word query — 'error' in verbose_logs.txt

```
./analyzer --file verbose_logs.txt --version v1 --buffer 512 --query word --word error
Indexing verbose_logs.txt -> version 'v1' ... Unique words: 439301
+-------------------------------------+ | WORD QUERY RESULT |
+-------------------------------------+ Version : v1 Word : error Count : 109702 Execution
time : 0.843503 seconds
```

7.2 Top-10 query — verbose_logs.txt

```
./analyzer --file verbose_logs.txt --version v1 --buffer 512 --query top --top 10 Unique
words: 439301 Rank Word Count ------------------------------------------------- 1 the
1647203 2 in 768309 3 a 548882 4 devops 548882 5 architecture 439217 6 at 439217 7
bottleneck 439217 8 causing 439217 9 internal 439217 10 module 439217 Execution time :
1.012016 seconds
```

7.3 Diff query — 'error' across both files

```
./analyzer --file1 verbose_logs.txt --version1 v1 --file2 test_logs.txt --version2 v2
--buffer 512 --query diff --word error Indexing verbose_logs.txt -> version 'v1' ... Unique
words: 439301 Indexing test_logs.txt -> version 'v2' ... Unique words: 33
+-------------------------------------+ | DIFF QUERY RESULT |
+-------------------------------------+ Version 1 : v1 Version 2 : v2 Word : error Count
(v1) : 109702 Count (v2) : 605079 Difference : +495377 Execution time : 1.477464 seconds
```

7.4 What the results show

The difference in unique word counts between the two files (439k vs 33) makes sense given their formats. The verbose file has full sentences with varied vocabulary, while the test file just cycles through a small fixed set of words. The word 'error' is more frequent in the test file because log levels account for a larger fraction of its total tokens. Both files were processed well under 2 seconds using a 512 KB buffer.

8. Conclusion

The indexer handles both 109 MB files correctly and efficiently, using a 512 KB buffer throughout. The design ended up fairly clean — each class does one thing, the query hierarchy makes it easy to add new query types later, and the buffer boundary handling in the Tokenizer was the most interesting part to get right. The program runs in about 1-2 seconds on the provided datasets with no memory issues.